

Task 1: Set Performance and Storage in Python vs. C++

Introduction

A set is a data structure used to store unique elements and perform fast membership testing. Python and C++ provide different types of sets, and each implementation has different performance characteristics.

Python Set

Python's built-in `set` is implemented using a **hash table**. Each element is processed using a hash function, and the resulting hash determines where the element is stored in the table.

For example:

```
numbers = {10, 20, 30, 40}

print(20 in numbers)
numbers.add(50)
numbers.remove(30)
```

Average time complexity:

Operation	Python set
-----------	------------

Search	$O(1)$ average
Insert	$O(1)$ average
Delete	$O(1)$ average
Worst case	$O(n)$

Python sets are usually very fast for membership testing because hashing allows the program to directly locate the approximate position of an element instead of searching every element.

C++ `std::set`

C++ `std::set` is normally implemented using a **balanced binary search tree**, commonly a Red-Black Tree. Elements are automatically sorted.

```
#include <set>
using namespace std;

set<int> numbers;

numbers.insert(10);
```

```
numbers.insert(20);
numbers.insert(30);

if (numbers.find(20) != numbers.end())
    cout << "Found";
```

Its main operations have:

- Search: $O(\log n)$
- Insert: $O(\log n)$
- Delete: $O(\log n)$

The advantage is that elements remain sorted.

C++ `std::unordered_set`

`std::unordered_set` uses a **hash table**, making it conceptually closer to Python's set.

```
#include <unordered_set>
using namespace std;

unordered_set<int> numbers;

numbers.insert(10);
numbers.insert(20);

if (numbers.find(20) != numbers.end())
    cout << "Found";
```

Average complexity:

- Search: $O(1)$
- Insert: $O(1)$
- Delete: $O(1)$

Worst case can become $O(n)$ because of hash collisions.

Hashing and Collisions

A hash function converts an element into a numerical hash value. The hash is used to determine a location, or **bucket**, in the hash table.

A collision occurs when multiple elements are assigned to the same bucket. Hash tables handle collisions using different techniques, and they may resize when they become too full.

Memory and Cache Behavior

Hash tables usually require extra memory for buckets and unused capacity. Tree-based sets require memory for tree nodes, pointers, and balancing information.

Hash tables may provide good average lookup performance, but their memory accesses can be less predictable. Tree structures also involve pointer-based memory access and may have poor cache locality.

Contiguous or compact hash-table layouts can often provide better practical performance because modern CPUs benefit from cache-friendly memory access.

Comparison

Feature	Python <code>set</code>	C++ <code>std::set</code>	C++ <code>std::unordered_set</code>
Structure	Hash table	Balanced tree	Hash table
Ordering	No	Sorted	No
Average search	$O(1)$	$O(\log n)$	$O(1)$
Insert	$O(1)$	$O(\log n)$	$O(1)$
Delete	$O(1)$	$O(\log n)$	$O(1)$
Memory	High	High	High
Main advantage	Fast membership	Sorted data	Fast unordered lookup

Conclusion

Use Python `set` when fast membership testing and removing duplicates are required. Use C++ `std::unordered_set` when fast average lookup is more important than ordering. Use C++ `std::set` when sorted elements and ordered traversal are required.

Task 2: JSON vs. TOON Data Formats

Introduction

JSON stands for **JavaScript Object Notation**. It is a widely used text-based data format for storing and exchanging structured information.

TOON stands for **Token-Oriented Object Notation**. It is a newer serialization format designed particularly to represent structured data in a compact, human-readable form and to reduce token usage when data is sent to Large Language Models (LLMs).

JSON Structure

JSON uses objects, arrays, keys, and values.

Example:

```
{
  "students": [
    {
      "name": "Ali",
      "age": 20,
      "grade": 90
    },
    {
      "name": "Sara",
      "age": 21,
      "grade": 95
    }
  ]
}
```

JSON is easy for programs to parse and is supported by almost every programming language.

TOON Structure

TOON represents structured data using a more compact notation, especially useful when many records have the same fields.

Conceptually, the same information can be represented as:

```
students[2]{name,age,grade}:
  Ali,20,90
  Sara,21,95
```

Instead of repeating "name", "age", and "grade" for every object, the field names can be defined once.

Comparison

Feature	JSON	TOON
Readability	Very good	Very good for tabular data
Compatibility	Excellent	More limited
File size	Usually larger	Can be smaller
Token usage	Higher	Often lower
LLM use	Good	Designed to be token-efficient
Tool support	Extremely wide	Newer ecosystem
Standardization	Very mature	Newer

Token Efficiency

JSON repeatedly includes property names, quotation marks, commas, and braces. TOON can reduce this repetition when a dataset contains many records with the same structure.

For example, a large list of students may require the field names to be repeated in JSON, while TOON can define the fields once.

Therefore, TOON can be useful for applications where **LLM token consumption and prompt size** are important.

Advantages of JSON

- Very widely supported.
- Easy to parse.
- Works with APIs and web applications.
- Human-readable.
- Mature tooling and libraries.
- Suitable for complex nested data.

Disadvantages of JSON

- Can become verbose.
- Repeated keys increase file size.
- Can require more LLM tokens.

Advantages of TOON

- Compact representation.
- Can reduce repeated structural information.
- Useful for structured/tabular data.
- Potentially reduces LLM token consumption.

Disadvantages of TOON

- Newer and less widely supported.
- JSON has much better compatibility.
- Existing APIs and applications commonly expect JSON.

Conclusion

JSON should be preferred for APIs, web applications, databases, configuration files, and systems requiring maximum compatibility. TOON can be preferred when structured data is sent to LLMs and reducing token usage is an important goal.

Task 3: Map vs. Dictionary in Python and C++

Introduction

A map or dictionary stores data as **key-value pairs**. The key is used to find the corresponding value.

Python provides `dict`, while C++ provides `std::map` and `std::unordered_map`.

Python Dictionary

Python's `dict` is implemented using a **hash table**.

```
student = {
    "name": "Ali",
    "age": 20,
    "grade": 90
}

print(student["name"])
student["grade"] = 95
```

Average complexity:

- Lookup: $O(1)$
- Insertion: $O(1)$
- Deletion: $O(1)$

Worst-case hash-table operations can be $O(n)$.

Python dictionaries preserve **insertion order**, but they are not sorted by key.

C++ `std::map`

C++ `std::map` is normally implemented using a balanced binary search tree, usually a Red-Black Tree.

```
#include <map>

map<string, int> students;

students["Ali"] = 90;
students["Sara"] = 95;
```

Complexities:

- Lookup: $O(\log n)$
- Insert: $O(\log n)$
- Delete: $O(\log n)$

Keys are maintained in sorted order.

C++ `std::unordered_map`

`std::unordered_map` uses a hash table.

```
#include <unordered_map>

unordered_map<string, int> students;

students["Ali"] = 90;
students["Sara"] = 95;
```

Average complexity:

- Lookup: $O(1)$
- Insert: $O(1)$
- Delete: $O(1)$

The elements are not sorted.

Comparison

Feature	Python dict	C++ <code>std::map</code>	C++ <code>std::unordered_map</code>
Structure	Hash table	Balanced tree	Hash table
Average lookup	$O(1)$	$O(\log n)$	$O(1)$
Insertion	$O(1)$	$O(\log n)$	$O(1)$
Deletion	$O(1)$	$O(\log n)$	$O(1)$
Ordering	Insertion order	Sorted	No ordering
Main advantage	Fast and flexible	Sorted keys	Very fast lookup

Memory Usage

Hash tables require bucket storage and extra capacity. Tree-based maps allocate separate nodes containing keys, values, pointers, and balancing information.

C++ containers can sometimes have lower overhead than Python dictionaries because Python objects themselves contain significant runtime metadata.

Practical Performance

For simple key lookup, Python `dict` and C++ `unordered_map` are usually much faster than tree-based maps because they have average $O(1)$ lookup.

However, `std::map` is useful when sorted order is required.

Conclusion

Use Python `dict` for general-purpose key-value storage in Python. Use `std::unordered_map` in C++ when fast average lookup is the priority. Use `std::map` when sorted keys and ordered operations are required.

Task 4: Amortized Analysis

Introduction

Amortized analysis studies the average cost of operations over a sequence of operations. It does not mean taking the normal statistical average. Instead, it guarantees that the total cost of a sequence is small, even if some individual operations are expensive.

Types of Analysis

Best Case

The minimum possible running time.

Worst Case

The maximum running time of a single operation.

Average Case

The expected running time based on an assumed distribution of inputs.

Amortized Case

The average cost of operations over a sequence, providing a guarantee for the entire sequence.

Dynamic Array Example

Suppose a dynamic array has capacity 4.

When the array becomes full, it creates a larger array and copies the existing elements.

For example:

```
Capacity 4:  
[1][2][3][4]
```

```
Add 5:  
[1][2][3][4] -> copy -> [1][2][3][4][5]
```

The resize operation costs $O(n)$, but it does not happen every time.

If capacity is doubled, the total number of copied elements over many insertions is:

$$1 + 2 + 4 + 8 + \dots + n$$

This geometric series is $O(n)$.

For n insertions, the total cost is $O(n)$, so the amortized cost per insertion is:

$$O(n) / n = O(1)$$

Therefore, dynamic-array insertion is **$O(1)$ amortized**.

Aggregate Method

The Aggregate Method calculates the total cost of a sequence and divides it by the number of operations.

Example:

$$\begin{aligned} n \text{ insertions} &= O(n) \text{ total} \\ \text{Amortized cost} &= O(n) / n = O(1) \end{aligned}$$

Accounting Method

The Accounting Method assigns an artificial cost to each operation.

Cheap operations are charged slightly more than their actual cost. The extra amount is stored as credit and later used to pay for expensive operations.

Potential Method

The Potential Method assigns a potential value to the data structure.

The amortized cost is:

$$\text{Amortized Cost} = \text{Actual Cost} + \text{Change in Potential}$$

The stored potential represents work that has effectively been prepaid for future operations.

Other Examples

- Dynamic arrays: $O(1)$ amortized insertion.
- Stack push: $O(1)$ amortized.
- Stack pop: $O(1)$.
- Queue operations: usually $O(1)$ amortized when implemented using suitable structures.
- Hash tables: resizing can be expensive, but insertion is generally $O(1)$ amortized.

Importance

Amortized analysis is important because looking only at the worst-case cost of every individual operation can give a misleading picture.

A dynamic array insertion may occasionally cost $O(n)$, but a long sequence of insertions can still have $O(1)$ amortized cost per operation.

Conclusion

Amortized analysis helps evaluate the real performance of data structures over long sequences of operations and explains why occasional expensive operations do not necessarily make the overall algorithm inefficient.

Task 5: BFS vs. DFS and Graph Traversal Algorithms

Introduction

Graph traversal means visiting the vertices of a graph systematically. The two most important traversal algorithms are **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.

Graphs can be represented using an adjacency matrix or adjacency list.

Adjacency Matrix

An adjacency matrix uses a 2D array.

If an edge exists between two vertices, the corresponding position contains 1; otherwise it contains 0.

Space complexity:

$$O(V^2)$$

It provides $O(1)$ edge checking but requires large memory for sparse graphs.

Adjacency List

An adjacency list stores the neighbors of every vertex.

Example:

```
A: B, C
B: A, D
C: A, D
D: B, C
```

Space complexity:

$$O(V + E)$$

It is usually better for sparse graphs.

BFS

BFS explores the graph level by level and uses a **queue**.

Python example:

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
```

```

while queue:
    node = queue.popleft()
    print(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

```

BFS is useful for finding the shortest path in an **unweighted graph**.

DFS

DFS explores as deeply as possible before backtracking. It can use recursion or a stack.

```

def dfs(graph, node, visited):
    visited.add(node)
    print(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

```

BFS vs. DFS

Feature	BFS	DFS
Main structure	Queue	Stack/recursion
Traversal	Level by level	Deep first
Time	$O(V + E)$	$O(V + E)$
Space	$O(V)$	$O(V)$
Shortest unweighted path	Yes	Not guaranteed
Cycle detection	Yes	Yes
Connected components	Yes	Yes
Topological sorting	No	Yes
Maze/search problems	Good	Good

Applications

BFS

- Shortest path in unweighted graphs.
- Social network levels.
- Finding nearby nodes.
- Connected components.

DFS

- Cycle detection.
- Topological sorting.
- Maze solving.
- Connected components.
- Exploring all possible paths.

Example

Consider:

```
A
 / \
B   C
 |   |
D   E
```

BFS from A:

A → B → C → D → E

DFS from A may produce:

A → B → D → C → E

The exact order can depend on the order in which neighbors are stored.

Conclusion

Use BFS when the problem involves levels or shortest paths in an unweighted graph. Use DFS when deep exploration, cycle detection, topological sorting, or recursive graph processing is required.

Task 6: Comparison of Shortest Path Algorithms

Introduction

Shortest-path algorithms find the minimum-cost path between vertices in a graph. The appropriate algorithm depends on whether the graph is weighted, whether negative edges exist, and whether a heuristic is available.

1. BFS

BFS is used for **unweighted graphs**, or graphs where every edge has the same cost.

Time complexity:

$$O(V + E)$$

It finds the shortest path in terms of the number of edges.

2. Dijkstra's Algorithm

Dijkstra's algorithm finds shortest paths from one source when edge weights are **non-negative**.

It repeatedly selects the vertex with the smallest known distance and relaxes its neighboring edges.

With a binary heap:

$$O((V + E) \log V)$$

It is commonly used in road networks and routing systems with non-negative costs.

3. Bellman-Ford

Bellman-Ford can handle graphs containing **negative edge weights**.

It repeatedly relaxes every edge.

Time complexity:

$$O(VE)$$

Its major advantage is that it can detect negative cycles.

4. Floyd-Warshall

Floyd-Warshall calculates shortest paths between **all pairs of vertices**.

Its time complexity is:

$$O(V^3)$$

Space complexity:

$$O(V^2)$$

It is suitable for relatively small graphs where shortest paths between every pair of vertices are required.

5. A*

A* is a heuristic search algorithm. It uses:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost from the start to the current node.
- $h(n)$ = estimated cost from the current node to the goal.
- $f(n)$ = estimated total path cost.

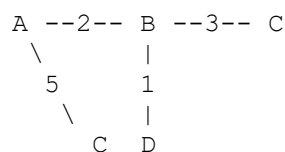
A* is widely used in games, robotics, maps, and navigation.

Comparison Table

Algorithm	Graph Type	Negative Edges	Negative Cycles	Time Complexity	Main Use
BFS	Unweighted	No	No	$O(V+E)$	Unweighted shortest path
Dijkstra	Weighted	No	No	$O((V+E)\log V)$	Positive-weight graphs
Bellman-Ford	Weighted	Yes	Detects them	$O(VE)$	Negative edges
Floyd-Warshall	Weighted	Yes	Can detect	$O(V^3)$	All-pairs shortest paths
A*	Weighted	Usually non-negative	No	Depends on heuristic	Goal-directed search

Example

Consider a weighted graph:



If the graph is unweighted, BFS is usually the simplest solution.

If the graph has positive weights, Dijkstra is usually preferred.

If negative edges exist, Bellman-Ford may be required.

If shortest paths between every pair of vertices are required and the graph is relatively small, Floyd-Warshall is appropriate.

If the goal is a specific destination and a good heuristic is available, A* can be faster than exploring the graph blindly.

Recommendations

- **Use BFS** for unweighted shortest-path problems.
- **Use Dijkstra** for weighted graphs with non-negative edges.
- **Use Bellman-Ford** when negative edge weights are possible.
- **Use Floyd-Warshall** for all-pairs shortest paths on smaller graphs.
- **Use A*** for goal-directed problems such as maps, games, and robotics.